

Pgm8

1. Install packages — only once

Open VS Code terminal in your project:

```
cd "C:\Users\Shamanth P\Desktop\EMPLOYEE CRUD OPERATIONS"
```

Then:

```
npm install express mongodb
```

You don't need to run this every time.

2. Start MongoDB

Terminal 1

Run:

```
cd "C:\Program Files\MongoDB\Server\8.3\bin"
```

Then:

```
.\mongod.exe --dbpath C:\data\db
```

Wait for:

Waiting for connections

Keep this terminal open.

3. Start your Node.js server

Terminal 2

Open a new terminal in VS Code.

Run:

```
cd "C:\Users\Shamanth P\Desktop\EMPLOYEE CRUD OPERATIONS"
```

Then:

```
node server.js
```

You should see:

Connected to MongoDB

Server running at http://localhost:3000

Keep this terminal open too.

```
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
const PORT = 3000;

app.use(express.json());

// MongoDB Connection
const MONGO_URI = "mongodb://127.0.0.1:27017";
const DB_NAME = "companyDB";

let employeesCollection;

async function connectDB() {
  try {
    const client = new MongoClient(MONGO_URI);

    await client.connect();
    console.log("Connected to MongoDB");

    const db = client.db(DB_NAME);
    employeesCollection = db.collection("employees");
  } catch (err) {
    console.log("Database Connection Error:", err);
  }
}

connectDB();

// ===== CREATE =====
```

```
app.post("/employees", async (req, res) => {  
  try {  
    const { employeeID, name, designation } = req.body;  
  
    const existing = await employeesCollection.findOne({  
      employeeID: String(employeeID)  
    });  
  
    if (existing) {  
      return res.status(400).json({  
        message: "Employee ID already exists"  
      });  
    }  
  
    const result = await employeesCollection.insertOne({  
      employeeID: String(employeeID),  
      name,  
      designation  
    });  
  
    res.status(201).json({  
      message: "Employee Added Successfully",  
      insertedId: result.insertedId  
    });  
  } catch (err) {  
    res.status(500).json({  
      message: err.message  
    });  
  }  
});
```

```
// ===== READ ALL =====
```

```
app.get("/employees", async (req, res) => {  
  try {  
    const employees = await employeesCollection  
      .find()  
      .toArray();  
  
    res.json(employees);  
  } catch (err) {  
    res.status(500).json({  
      message: err.message  
    });  
  }  
});
```

```
// ===== READ ONE =====
```

```
app.get("/employees/:id", async (req, res) => {  
  try {  
    const employee = await employeesCollection.findOne({  
      employeeID: String(req.params.id)  
    });  
  
    if (!employee) {  
      return res.status(404).json({  
        message: "Employee Not Found"  
      });  
    }  
  
    res.json(employee);  
  } catch (err) {  
    res.status(500).json({
```

```

        message: err.message
    });
}
});

// ===== UPDATE =====

app.put("/employees/:id", async (req, res) => {
    try {
        const result = await employeesCollection.updateOne(
            {
                employeeID: String(req.params.id)
            },
            {
                $set: req.body
            }
        );

        if (result.matchedCount === 0) {
            return res.status(404).json({
                message: "Employee Not Found"
            });
        }

        res.json({
            message: "Employee Updated Successfully"
        });
    } catch (err) {
        res.status(500).json({
            message: err.message
        });
    }
}

```

```

});

// ===== DELETE =====

app.delete("/employees/:id", async (req, res) => {
  try {
    const result = await employeesCollection.deleteOne({
      employeeID: String(req.params.id)
    });

    if (result.deletedCount === 0) {
      return res.status(404).json({
        message: "Employee Not Found"
      });
    }

    res.json({
      message: "Employee Deleted Successfully"
    });
  } catch (err) {
    res.status(500).json({
      message: err.message
    });
  }
});

// ===== START SERVER =====

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});

```